Software Requirements Specification

for

Multi-Model Network Intrusion Detection System (NIDS)

Version 1.0 approved

Prepared by Nilanjana Jamindar

RV University

27th April, 2026

Table of Contents

Revision History

Name	Date	Reason For Changes	Version
Nilanjana Jamindar	25th April 2026	Initial Draft	1.0
Nilanjana Jamindar	27th April 2026	Removed QNN model requirement	1.1

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete functional and non-functional requirements for the Full Stack Based Network Intrusion Detection System (NIDS). This system leverages an ensemble of classical machine learning algorithms, deep neural networks, and quantum machine learning models to detect network anomalies and intrusions with high accuracy.

This document covers Version 1.0 of the system and serves as the authoritative reference for design, development, testing, and deployment activities. It is applicable to the entire system, from data ingestion and model training to the web-based interactive dashboard.

1.2 Document Conventions

The following conventions are used throughout this document:

- Bold text indicates section headers, subheadings, bullet point headings or key terms being introduced for the first time.
- Italicized text indicates field names, filenames, or technical identifiers (e.g., *Train_data.csv, app.py*).
- REQ-X notation is used to uniquely identify individual functional requirements.
- Priority levels are indicated as: High, Medium, or Low.
- All requirements written in this SRS are inherited by sub-components unless explicitly overridden.

1.3 Intended Audience and Reading Suggestions

This document is intended for the following audiences:

- **Developers and Engineers:** Focus on Sections 3, 4, and 5 for technical requirements and system feature specifications.
- **Project Managers:** Begin with Section 1 (Introduction) and Section 2 (Overall Description) for scope and constraints.
- **Testers and QA Engineers:** Sections 4 and 5 define the functional and non-functional requirements to be validated.
- **Research Supervisors/Reviewers:** Sections 1, 2, and 4 provide the research motivation, architecture, and feature set.
- **End Users (Security Analysts):** Section 3 (External Interface Requirements) describes the user-facing dashboard interface.

It is recommended to read this document sequentially. Readers interested in implementation details should focus on Sections 4 and 5.

1.4 Product Scope

The Full Stack Network Intrusion Detection System is a research-grade, end-to-end cybersecurity platform designed to classify network traffic as normal or anomalous using multiple AI paradigms. The system includes the following key components:

- **Data Preprocessing Pipeline:** Encoding categorical features (protocol_type, service, flag), standard scaling, and PCA-based dimensionality reduction for quantum models.
- **Classical ML Module:** Six classical classifiers — K-Nearest Neighbors (KNN), Decision Tree, Random Forest, Support Vector Machine (SVM), Naive Bayes, and XGBoost.
- **Deep Learning Module:** A fully connected Deep Neural Network (DNN) with dropout regularization, trained using binary cross-entropy.
- **Quantum ML Module:** Quantum Support Vector Machine (QSVM) and Variational Quantum Classifier (VQC) implemented using Qiskit and Qiskit Machine Learning.
- **Streamlit Web Dashboard:** A real-time interactive dashboard that accepts user input for network features and returns live predictions from all models, along with comparative accuracy charts.

The system is evaluated on the KDD Cup 1999 dataset (*Train_data.csv*: 25,192 records, 41 features + class label). The primary goal is to demonstrate the effectiveness of quantum-enhanced machine learning in cybersecurity intrusion detection and provide a comparative benchmark against classical and deep learning approaches.

1.5 References

1. US Air Force LAN — Kaggle Dataset: https://www.kaggle.com/datasets/sampadab17/network-intrusion-detection
2. Qiskit Documentation — IBM Quantum: https://qiskit.org/documentation/
3. Qiskit Machine Learning — https://qiskit-community.github.io/qiskit-machine-learning/
4. Streamlit Documentation — https://docs.streamlit.io/
5. Scikit-learn Documentation — https://scikit-learn.org/stable/
6. TensorFlow/Keras Documentation — https://www.tensorflow.org/
7. IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications
8. Wiegers, K.E. (1999). Software Requirements Specification Template. Process Impact.

2. Overall Description

2.1 Product Perspective

The Full Stack NIDS is a self-contained, research-oriented application that extends conventional intrusion detection approaches by integrating quantum machine learning into the classification pipeline. It is not a replacement for production-grade security tools (e.g., Snort, Suricata), but rather a proof-of-concept platform demonstrating how quantum computing paradigms can augment AI-based threat detection.

The system interfaces with static CSV datasets (*Train_data.csv* and *Test_data.csv*) for offline training. It interacts with the Qiskit runtime for quantum circuit execution and Streamlit's web server for the user-facing dashboard. Saved model artifacts (*.pkl*, *.keras*) are used to serve predictions in the web interface without retraining.

The system's architecture can be summarized as: Raw CSV Data → Preprocessing → Model Training (Classical / Deep / Quantum) → Serialization → Streamlit Dashboard → Real-Time Prediction Display.

2.2 Product Functions

The major functions of the system are:

- Data ingestion from CSV files (*Train_data.csv* and *Test_data.csv*) using pandas.
- Categorical encoding of *protocol_type*, *service*, and *flag* features using LabelEncoder.
- Feature normalization using StandardScaler (for classical/DNN models) and MinMaxScaler + PCA (for quantum models).
- Training and evaluation of 6 classical ML classifiers with accuracy and classification reports.
- Training of a Deep Neural Network with binary cross-entropy loss and Adam optimizer.
- Training of QSVM (ZZFeatureMap, 4 qubits) and VQC (RealAmplitudes ansatz, COBYLA optimizer) on 100-sample quantum-compatible subsets.
- Comparison of all model accuracies using a tabular view with precision, recall, and F1-score.
- Serialization of all models, scalers, and encoders to disk using joblib, dill, and .keras formats.
- A Streamlit web dashboard for real-time intrusion detection with live predictions from all model categories.

2.3 User Classes and Characteristics

The following user classes are identified for this system:

- **Security Analysts (Primary Users):** Monitor network activity through the dashboard. Expected to have basic IT knowledge; no ML expertise required. Interact only with the Streamlit UI.
- **Machine Learning Researchers:** Modify model architectures, hyperparameters, and quantum circuits. Expected to have strong Python, Qiskit, and scikit-learn expertise.
- **System Administrators:** Deploy and maintain the Streamlit application. Expected to have Linux/cloud deployment knowledge.
- **Academic Supervisors/Reviewers:** Evaluate system design and performance. Interact with the documentation and results tables.

2.4 Operating Environment

The system is designed to operate in the following environment:

- **Primary Environment:** Google Colab (Python 3.x, GPU-enabled runtime recommended).
- **Operating System:** Linux (Ubuntu, via Colab runtime) or Windows 10/11 locally.
- **Python Version:** 3.8 or higher.

- **Key Libraries:** pandas, numpy, scikit-learn, TensorFlow/Keras (>=2.x), XGBoost, Qiskit, qiskit-machine-learning, qiskit-algorithms, streamlit, pyngrok, joblib, dill.
- **Dashboard Hosting:** Local Streamlit server exposed via pyngrok tunnel (port 8501).
- **Hardware:** Minimum 8GB RAM for classical models; 16GB+ RAM recommended for quantum model training. No dedicated GPU required (though beneficial for DNN training).

2.5 Design and Implementation Constraints

- **Quantum Model Scalability:** Quantum models (QSVM and VQC) are trained on a subset of 100 training samples and 50 validation samples due to exponential simulation complexity on classical hardware. This is a fundamental constraint of quantum simulation.
- **PCA Dimensionality:** Input to quantum models is reduced to 4 dimensions using PCA. This limits the feature information available for quantum classification.
- **Stateless Dashboard:** The Streamlit app loads pre-trained models on startup; no real-time retraining is supported in the deployed dashboard.
- **Input Feature Coverage:** The dashboard exposes only 10 of 41 features for user input. Remaining features are set to zero, which may affect prediction accuracy for certain attack types.
- **Binary Classification Only:** The system classifies traffic as either normal or anomaly (binary), not multi-class attack type classification.
- **Network Dependency:** The pyngrok tunnel requires an active internet connection and a valid ngrok authentication token.

2.6 User Documentation

- This SRS document (primary technical reference).
- Jupyter Notebook (Full_Stack_based_Network_Intrusion_Detection.ipynb) with inline code comments and output cells.
- Streamlit Dashboard inline help: tooltips and metric labels within the web interface.
- README file: step-by-step instructions for installing dependencies, running the notebook, and launching the dashboard.

2.7 Assumptions and Dependencies

- The *Train_data.csv* and *Test_data.csv* files conform to the KDD Cup 1999 dataset schema with 42 columns (41 features + class label).
- The target column is named 'class' with values 'normal' and 'anomaly'.
- Qiskit and qiskit-machine-learning packages are compatible with the installed Python and numpy versions.
- A valid ngrok authentication token is available for public dashboard exposure.
- The Google Colab environment provides pip installation access for all listed dependencies.
- The ZZFeatureMap and RealAmplitudes Qiskit circuit components maintain backward-compatible APIs.

3. External Interface Requirements

3.1 User Interfaces

The primary user interface is the Streamlit web application (*app.py*), accessible via a browser through the ngrok-generated public URL. The interface includes:

- **Sidebar Panel:** Contains 10 interactive input controls for network traffic feature selection — dropdown menus for *protocol_type*, *service*, and *flag*; number input fields for *src_bytes* and *dst_bytes*; and sliders for *count*, *srv_count*, *same_srv_rate*, *diff_srv_rate*, and *dst_host_srv_count*.
- **Main Panel — Live Predictions:** Displays four metric cards (Best ML Model, DNN, QSVM, VQC), each showing NORMAL or ANOMALY with color-coded indicators.
- **Main Panel — Accuracy Table:** A sortable dataframe listing all 9 model algorithms with their accuracy scores, loaded dynamically from *accuracies.pkl*.
- **Main Panel — Bar Chart:** A comparative bar chart visualization of model accuracy scores using Streamlit's built-in charting.
- **Page Title:** 'Multi-Model Intrusion Detection' with shield emoji for visual clarity.

The interface follows a wide layout (*layout='wide'*) for optimal display on standard widescreen monitors.

3.2 Hardware Interfaces

The system does not interface directly with any network hardware or packet capture devices. All input data is provided through CSV files (offline mode) or manual feature entry via the dashboard. For production deployment, integration with network TAPs or packet brokers would be required, but this is outside the scope of the current version.

The quantum models require access to a classical CPU for Qiskit's quantum circuit simulation (Aer simulator). No quantum hardware (QPU) interface is required in this version.

3.3 Software Interfaces

- *pandas (>=1.3)*: DataFrame operations for data ingestion and preprocessing.
- *scikit-learn (>=0.24)*: LabelEncoder, StandardScaler, MinMaxScaler, PCA, train_test_split, and all classical ML classifiers.
- *TensorFlow/Keras (>=2.8)*: Deep Neural Network model construction, training, and inference.
- *XGBoost (>=1.5)*: Gradient boosting classifier for the XGBoost model.
- *Qiskit (>=0.43)*: Quantum circuit construction (ZZFeatureMap, RealAmplitudes).
- *qiskit-machine-learning (>=0.5)*: QSVC and VQC classifiers.
- *qiskit-algorithms (>=0.2)*: COBYLA optimizer for VQC training.
- *joblib*: Serialization of scikit-learn models and NumPy arrays.
- *dill*: Serialization of Qiskit-based quantum models (which are incompatible with standard pickle).
- *Streamlit (>=1.10)*: Web application framework for the real-time dashboard.
- *pyngrok*: ngrok Python wrapper for tunneling the Streamlit server to a public URL.

3.4 Communications Interfaces

- **HTTP (Port 8501):** Streamlit's default web server protocol for browser-based dashboard access.
- **ngrok Tunnel:** Encrypted TLS tunnel from the local Streamlit server (localhost:8501) to a public ngrok URL. Requires HTTPS.
- **ngrok API (HTTPS):** Used by pyngrok to register and manage the public tunnel endpoint. Authentication via user-provided ngrok token.

4. System Features

4.1 Data Ingestion and Preprocessing

4.1.1 Description and Priority

This feature handles loading the KDD Cup 1999 dataset from CSV files, encoding categorical variables, scaling features, and preparing data splits for model training. **Priority: HIGH.**

4.1.2 Stimulus/Response Sequences

- User runs the notebook cell containing *pd.read_csv()* → System loads *Train_data.csv* (25,192 records, 42 columns) and *Test_data.csv* into pandas DataFrames.
- User triggers *preprocess_data()* → System applies LabelEncoder to *protocol_type*, *service*, and *flag*; encodes the class column (*anomaly=0, normal=1*); applies StandardScaler to all features; and returns the processed DataFrame along with encoder and scaler objects.
- User triggers *train_test_split()* → System splits data into 80% training (*X_train, y_train*) and 20% validation (*X_val, y_val*) sets with *random_state=42*.

4.1.3 Functional Requirements

- REQ-1: The system shall load *Train_data.csv* and *Test_data.csv* using pandas and validate that both contain 42 columns with the required column names.
- REQ-2: The system shall apply LabelEncoder to categorical columns: protocol_type, service, and flag. Unknown labels in the test set shall be mapped to -1.
- REQ-3: The system shall encode the target class column such that 'anomaly'=0 and 'normal'=1.
- REQ-4: The system shall apply StandardScaler (fit on training data only) and transform both training and validation sets.
- REQ-5: The system shall apply MinMaxScaler (*feature_range=[0,1]*) followed by PCA(*n_components=4*) for quantum model input preparation, fit on training data only.
- REQ-6: The quantum-compatible subset shall comprise the first 100 training samples and first 50 validation samples.

4.2 Classical Machine Learning Classification

4.2.1 Description and Priority

This feature implements training, evaluation, and serialization of six classical ML classifiers on the preprocessed KDD dataset. **Priority: HIGH.**

4.2.2 Stimulus/Response Sequences

- User runs the model training cell → System trains each of the 6 classifiers on (*X_train, y_train*) and evaluates on (*X_val, y_val*).
- System prints accuracy for each model and identifies the best-performing model by accuracy.
- System serializes the best ML model to *best_ml_model.pkl* using joblib.

4.2.3 Functional Requirements

- REQ-7: The system shall train and evaluate the following classifiers: *KNeighborsClassifier*, *DecisionTreeClassifier*, *RandomForestClassifier*, *SVC (probability=True)*, *GaussianNB*, and *XGBClassifier*.
- REQ-8: The system shall compute *accuracy_score* for each model on the validation set and store results in the *ml_results* dictionary.
- REQ-9: The system shall identify the model with the highest validation accuracy as *best_ml_name*.
- REQ-10: The system shall serialize the best ML model to *best_ml_model.pkl*.

4.3 Deep Neural Network Classification

4.1.1 Description and Priority

This feature implements a Keras-based deep neural network for binary intrusion detection classification. **Priority: HIGH.**

4.1.2 Stimulus/Response Sequences

- User calls *build_dnn(input_dim)* → System constructs a Sequential model with layers: Input(41) → Dense(64, ReLU) → Dropout(0.2) → Dense(32, ReLU) → Dense(1, Sigmoid).
- User calls *model.fit()* → System trains the DNN with Adam optimizer and binary_crossentropy loss.
- System generates predictions as probabilities; threshold 0.5 determines class label *(0=anomaly, 1=normal).*
- System saves the trained model to *dnn_model.keras* using the native Keras format.

4.1.3 Functional Requirements

- REQ-11: The DNN shall use the architecture: Input layer (41 features), Dense(64, relu), Dropout(0.2), Dense(32, relu), Dense(1, sigmoid).
- REQ-12: The model shall be compiled with *optimizer='adam'*, *loss='binary_crossentropy'*, metrics=['accuracy'].

- REQ-13: Predictions shall use a decision threshold of 0.5 (probability > 0.5 = normal; else anomaly).
- REQ-14: The trained model shall be saved in *.keras* format for compatibility with *tf.keras.models.load_model()*.

4.4 Quantum Machine Learning Classification

4.1.1 Description and Priority

This feature implements two quantum classifiers — QSVM using ZZFeatureMap and VQC using RealAmplitudes ansatz — via Qiskit Machine Learning. **Priority: MEDIUM**.

4.1.2 Stimulus/Response Sequences

- User triggers quantum preprocessing → System scales input to [0,1] using *MinMaxScaler* and reduces to 4 dimensions using PCA on a 100-sample subset.
- User triggers QSVM training → System creates *ZZFeatureMap (feature_dimension=4, reps=2, entanglement='linear')*, fits QSVC on training subset, and evaluates accuracy.
- User triggers VQC training → System creates VQC with ZZFeatureMap + RealAmplitudes ansatz, trains with *COBYLA(maxiter=100)* optimizer, and evaluates accuracy.
- Both models are serialized to *.pkl* files using dill.

4.1.3 Functional Requirements

- REQ-15: The QSVM shall use ZZFeatureMap with *feature_dimension=4, reps=2*, and linear entanglement.
- REQ-16: The VQC shall use ZZFeatureMap as feature map and RealAmplitudes(*num_qubits=4, reps=2*) as ansatz, trained with COBYLA optimizer (*maxiter=100*).
- REQ-17: Quantum models shall be trained on the first 100 training samples and validated on the first 50 validation samples.
- REQ-18: Both quantum models shall be serialized using dill to *qsvc_model.pkl* and *vqc_model.pkl*.
- REQ-19: Quantum model logging shall be suppressed to ERROR level to maintain clean output.

4.5 Model Comparison and Evaluation

4.1.1 Description and Priority

This feature aggregates performance metrics from all 9 models into a comparison table and visualization. **Priority: HIGH.**

4.1.2 Stimulus/Response Sequences

- User triggers the comparison cell → System collects accuracy, precision, recall, and F1-score (weighted average) for all models.

- System generates a sorted pandas DataFrame, highlights maximum-performing model per metric in green, and displays a styled table.
- System exports results to *model_comparison_results.csv* and saves *all_accuracies* dict to *accuracies.pkl*.

4.1.3 Functional Requirements

- REQ-20: The system shall compute accuracy, weighted precision, weighted recall, and weighted F1-score for all 9 models using sklearn.metrics.classification_report.
- REQ-21: The comparison DataFrame shall be sorted in descending order by accuracy.
- REQ-22: The system shall save the complete accuracy dictionary (all_accuracies) to *accuracies.pkl* for use by the Streamlit dashboard.
- REQ-23: Results shall be exported to *model_comparison_results.csv*.

4.6 Streamlit Real-Time Dashboard

4.1.1 Description and Priority

The Streamlit dashboard provides a browser-based interface for real-time intrusion prediction using pre-trained models. **Priority: HIGH**.

4.1.2 Stimulus/Response Sequences

- User accesses the ngrok URL → Browser loads the Streamlit dashboard; all models are loaded from *.pkl* and *.keras* files.
- User adjusts sidebar inputs (*protocol_type*, *src_bytes*, *count*, etc.) → Dashboard recomputes predictions for all 4 model categories and updates metric cards.
- Dashboard displays accuracy table and bar chart loaded from *accuracies.pkl*.

4.1.3 Functional Requirements

- REQ-24: The dashboard shall load *scaler.pkl*, *pca.pkl*, *encoders.pkl*, *accuracies.pkl*, *best_ml_model.pkl*, *dnn_model.keras*, *qsvc_model.pkl*, and *vqc_model.pkl* on startup using st.cache_resource.
- REQ-25: The sidebar shall provide interactive controls for: *protocol_type* (selectbox), *service* (selectbox), *flag* (selectbox), *src_bytes* (number_input), *dst_bytes* (number_input), *count* (slider 0-511), *srv_count* (slider 0-511), *same_srv_rate* (slider 0.0-1.0), *diff_srv_rate* (slider 0.0-1.0), and *dst_host_srv_count* (slider 0-255).
- REQ-26: The dashboard shall display four prediction metric cards: Best ML Model, DNN (Deep Learning), QSVM (Quantum), and VQC (Quantum).
- REQ-27: Each prediction shall be labeled as NORMAL (checkmark) or ANOMALY (alert) using clear visual indicators.
- REQ-28: The accuracy table shall be dynamically loaded from *accuracies.pkl* and displayed using *st.dataframe* with full container width.
- REQ-29: The bar chart shall display model accuracy comparison using *st.bar_chart*.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- **Classical ML Training Time:** All 6 classical models shall complete training within 5 minutes on a standard Colab CPU runtime for the 25,192-record dataset.
- **DNN Training Time:** The DNN shall complete training within 10 minutes for up to 50 epochs on a Colab CPU runtime.
- **Quantum Model Training Time:** QSVM and VQC shall complete training within 30 minutes on 100 samples using Qiskit's Aer statevector simulator.
- **Dashboard Response Time:** Streamlit dashboard predictions shall refresh within 3 seconds of any user input change for classical and DNN models. Quantum model predictions may have higher latency due to circuit simulation.
- **Memory Usage:** The system shall not exceed 16GB RAM during training on a Colab environment.

5.2 Safety Requirements

- The system processes only static, offline network traffic records. No real-time packet capture is performed, eliminating risks of traffic disruption.
- Predicted outputs are for informational and research purposes only. The system must not be used as the sole basis for automated network blocking decisions in production environments.
- Model predictions shall always be clearly labeled with the model type to prevent misattribution of results.

5.3 Security Requirements

- The ngrok public URL shall be kept confidential and not shared publicly during active sessions to prevent unauthorized access to the prediction interface.
- The ngrok authentication token (inserted in the notebook) shall not be committed to public version control repositories.
- Model serialization files (*.pkl*, *.keras*) shall be stored in a secure environment and not exposed publicly, as they may contain sensitive model weights.
- No user authentication is implemented in this version. In production deployment, OAuth or token-based authentication shall be added to the Streamlit dashboard.

5.4 Software Quality Attributes

- **Accuracy:** The ensemble of models shall achieve a minimum of 90% binary classification accuracy on the validation set for classical ML models and DNN, as benchmarked on the KDD Cup 1999 dataset.

- **Reliability:** The Streamlit dashboard shall successfully load all pre-trained model artifacts without errors on each startup, given that the *.pkl* and *.keras* files are present in the working directory.
- **Maintainability:** The preprocessing pipeline (preprocess_data function) shall be modular and reusable for both training and inference data paths. Model serialization shall use standard formats (joblib, dill, .keras) for cross-session persistence.
- **Portability:** The system shall run on Google Colab without modification. Local deployment on Linux/Windows with Python 3.8+ shall require only standard pip installations.
- **Extensibility:** The model dictionary (models = {...}) shall be designed to allow addition of new classifiers without modifying the evaluation loop.
- **Usability:** The Streamlit interface shall not require ML expertise for basic usage. All controls shall have descriptive labels and reasonable default values.

5.5 Business Rules

- The system is developed exclusively for academic and research purposes under the scope of the associated course/project.
- The KDD Cup 1999 dataset is used under open-access research terms and shall be cited in all publications or presentations.
- Model comparison results shall be presented objectively; no model shall be suppressed or excluded from the comparison table regardless of performance.
- Quantum models are evaluated on a reduced dataset subset only due to computational constraints and shall be explicitly noted as such in any performance comparisons.

6. Other Requirements

The following additional requirements apply to this project:

- **Dataset Integrity:** The *Train_data.csv* and *Test_data.csv* files must not be modified after the preprocessing stage. All transformations are applied in-memory using DataFrame copies.
- **Reproducibility:** The *train_test_split* function shall use *random_state=42* to ensure reproducible data splits across runs.
- **Logging:** Qiskit Machine Learning warnings shall be suppressed to logging.ERROR level to maintain clean notebook output.
- **Version Control:** All source files, including the Jupyter notebook and *app.py*, should be managed under Git version control.
- **Notebook Execution Order:** All cells must be executed sequentially. Out-of-order execution may cause NameError or missing artifact errors during dashboard deployment.

Appendix A: Glossary

Term / Acronym	Definition
NIDS	Network Intrusion Detection System — a system that monitors network traffic for suspicious activity.
KDD Cup 1999	A benchmark dataset from the Third International Knowledge Discovery and Data Mining Tools Competition, containing simulated network traffic labeled as normal or attack.
KNN	K-Nearest Neighbors — a non-parametric classification algorithm.
SVM / SVC	Support Vector Machine / Support Vector Classifier — a supervised learning algorithm for classification.
DNN	Deep Neural Network — a multi-layer artificial neural network.
QSVM	Quantum Support Vector Machine — a quantum-enhanced version of SVM using quantum kernel computation.
VQC	Variational Quantum Classifier — a parameterized quantum circuit trained using a classical optimizer.
PCA	Principal Component Analysis — dimensionality reduction technique.
Qiskit	An open-source SDK for working with quantum computers at the level of circuits, pulses, and algorithms (IBM).
COBYLA	Constrained Optimization By Linear Approximation — a gradient-free optimizer used for VQC training.
ZZFeatureMap	A quantum feature map that encodes classical data into quantum states using ZZ interactions.
RealAmplitudes	A parameterized quantum circuit ansatz used in the VQC model.
Streamlit	An open-source Python library for building interactive web applications for ML.
ngrok	A tool that creates secure tunnels to expose local servers to the public internet.
pyngrok	Python wrapper for ngrok.
joblib	A Python library for lightweight pipelining and model serialization.
dill	An extension of pickle that supports serialization of complex Python objects including Qiskit models.
XGBoost	Extreme Gradient Boosting — an optimized gradient boosting algorithm.
Binary Classification	A classification task with two possible output classes (here: normal vs. anomaly).
LabelEncoder	A scikit-learn utility for encoding categorical labels as integers.
StandardScaler	Scikit-learn preprocessor that standardizes features to zero mean and unit variance.
MinMaxScaler	Scikit-learn preprocessor that scales features to a specified range (default [0,1]).

Appendix B: Analysis Models

The following analysis model describes the data flow through the system:

- Stage 1 — Input: Train_data.csv (25,192 x 42), Test_data.csv. Protocol_type, service, and flag are categorical; all others are numerical.
- Stage 2 — Preprocessing: LabelEncoding → StandardScaling → train_test_split (80/20). For quantum: additionally MinMaxScaling → PCA(4 components) → sample to 100/50 records.
- Stage 3 — Classical ML Training: 6 classifiers trained in parallel on standardized features. Best model identified by validation accuracy.
- Stage 4 — DNN Training: Sequential network trained with binary cross-entropy. Predictions thresholded at 0.5.
- Stage 5 — Quantum ML Training: QSVC and VQC trained on PCA-reduced quantum subset. Accuracy computed on 50-sample quantum validation set.
- Stage 6 — Evaluation: All 9 models evaluated by accuracy, precision, recall, F1-score. Results tabulated and sorted.
- Stage 7 — Serialization: All models, scalers, encoders, and accuracies saved to disk.
- Stage 8 — Dashboard: Streamlit app loads artifacts, accepts user input for 10 features, and displays real-time predictions from Best ML, DNN, QSVM, and VQC models.

Appendix C: To Be Determined List

TBD #	Reference	Description
TBD-1	REQ-7	Optimal hyperparameters for KNN (k value), Random Forest (n_estimators), and SVM (C, kernel) to be determined via cross-validation.
TBD-2	REQ-11	Optimal DNN architecture (number of layers, units per layer, dropout rate) to be confirmed through hyperparameter search.
TBD-3	REQ-16	Optimal COBYLA maxiter value and VQC reps for convergence — to be determined through ablation study.
TBD-4	Section 3.1	Exact set of 10 input features exposed in the Streamlit sidebar — remaining 31 features currently set to zero; strategy for incorporating all 41 features is TBD.
TBD-5	Section 5.3	Authentication mechanism for the Streamlit dashboard in production deployment — OAuth, token-based, or other — TBD.
TBD-6	Section 4.4	Whether to extend quantum models to use real IBM Quantum hardware (QPU) via IBMQ account — TBD based on resource availability.